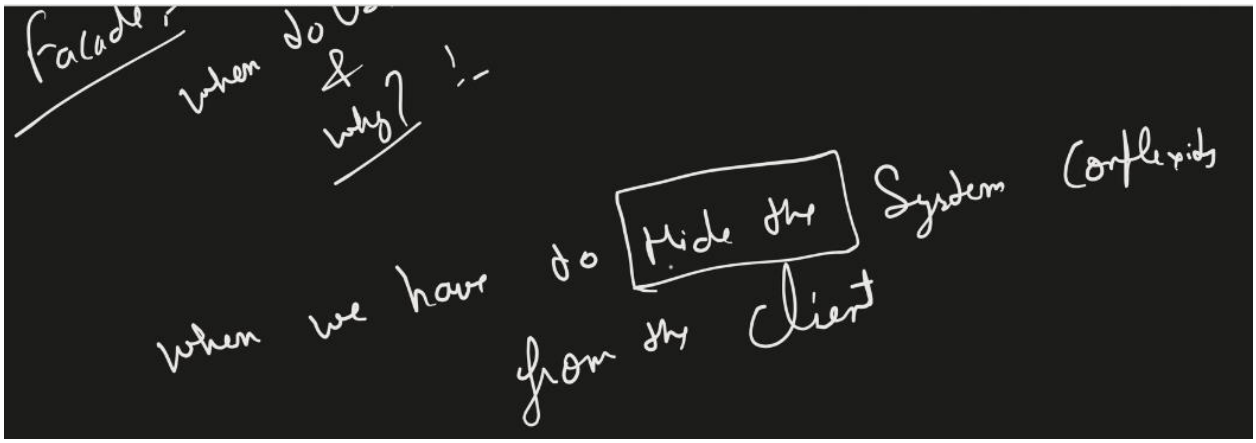
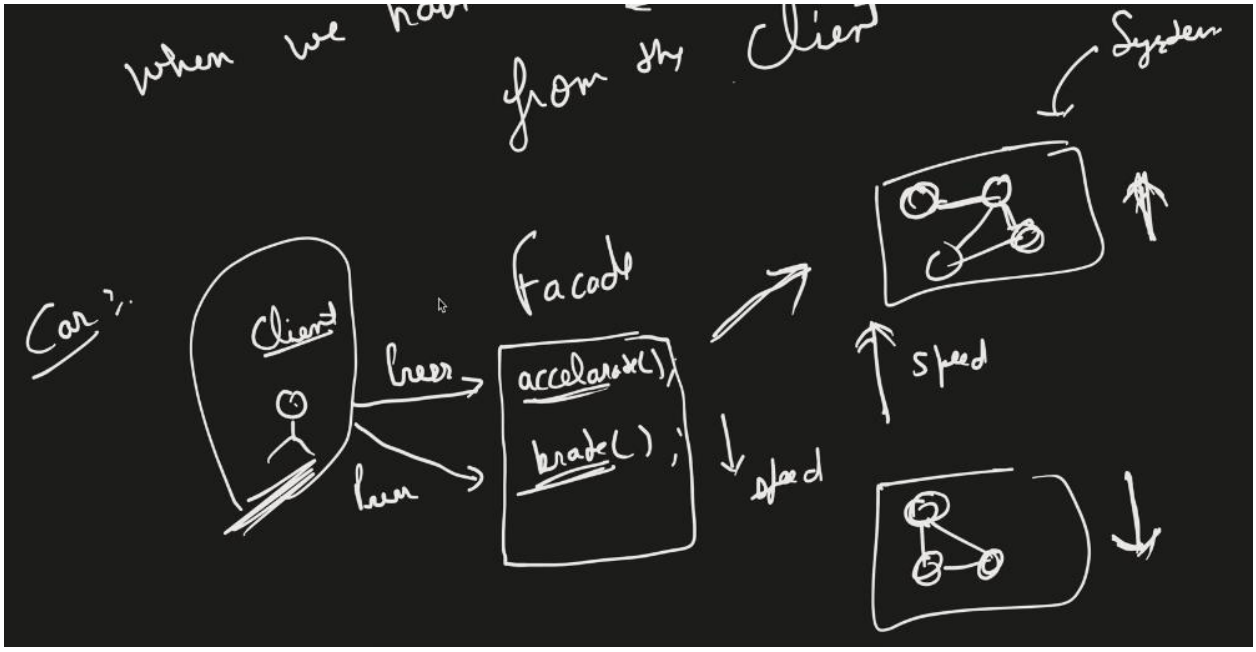
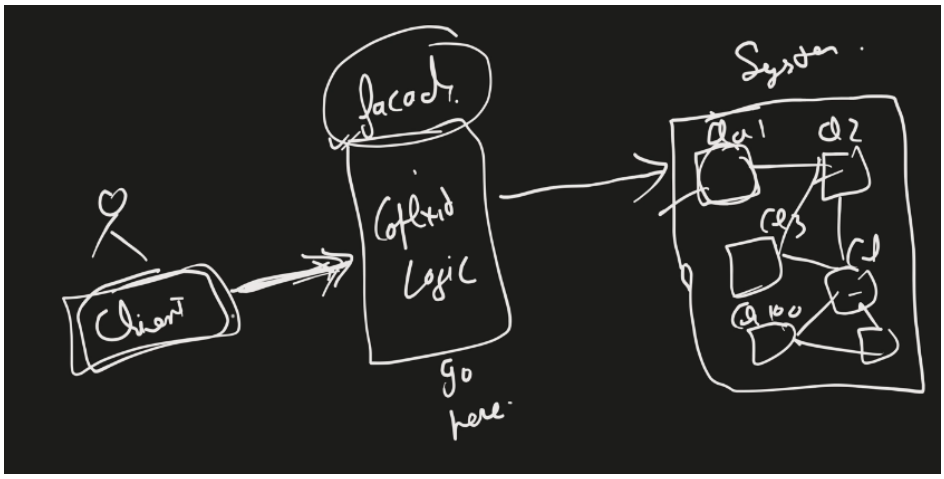
24. Facade Design Pattern (Structural Design Pattern)



The internal level system components interaction is hidden from the client





Case 1 :

Scenario 1:

```java
public class EmployeeClient {

    public void getEmployeeDetails() {
        EmployeeFacade employeeFacade = new EmployeeFacade();
        Employee employeeDetails = employeeFacade.getEmployeeDetails( empID: 121222);
    }
}
```

```java
public class EmployeeFacade {

    EmployeeDAO employeeDAO;

    public EmployeeFacade(){
        employeeDAO = new EmployeeDAO();
    }
```

```java
public class EmployeeDAO {

    public void insert(){
        //insert into Employee Table
    }

    public void updateEmployeeName(){
        //updating employee Name
```

DAO
(Data Access Object)

Employee DAO

Insert()
update Name()
update Address()
getED (EmpID)
getED (email id)
⋮
50 - 100

```java
public class EmployeeFacade {

    EmployeeDAO employeeDAO;

    public EmployeeFacade(){
        employeeDAO = new EmployeeDAO();
    }

    public void insert(){
        employeeDAO.insert();
    }

    public Employee getEmployeeDetails(int empID){
        return employeeDAO.getEmployeeDetails(empID);
    }
}
```

```java
public class EmployeeDAO {

    public void insert(){
        //insert into Employee Table
    }

    public void updateEmployeeName(){
        //updating employee Name
    }

    public Employee getEmployeeDetails(String emailID){
        //get employee details based on Emp ID
        return new Employee();
    }

    public Employee getEmployeeDetails(int empID){
        //get employee details based on Emp ID
        return new Employee();
    }
}
```

Case 2:

```java
public class ProductDAO {

    public Product getProduct(int productId){
        //get product based on product id and return
        return new Product();
    }
}
```

```java
public class Payment {

    public boolean makePayment(){
        //initiate payment and return true if success
        return true;
    }
}
```

```java
public class Invoice {

    public void generateInvoice(){
        //this will generate the invoice
    }
}
```

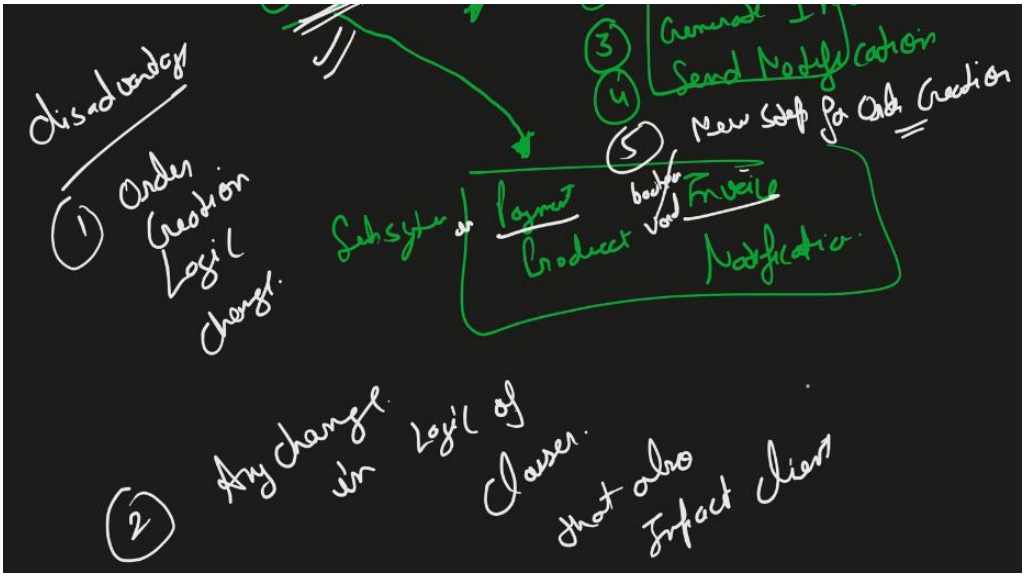
```java
public class SendNotification {

    public void sendNotification(){
        //this will send notification to customer on mobile
    }
```

Client

Create order

1. Get Product
2. Make Payment
3. Generate Invoice
4. Send Notification

Subsystem: Payment, Product, Invoice, Notification




```java
public class OrderClient {

    public static void main(String args[]){
        OrderFacade orderFacade = new OrderFacade();
        orderFacade.createOrder();
    }
}
```

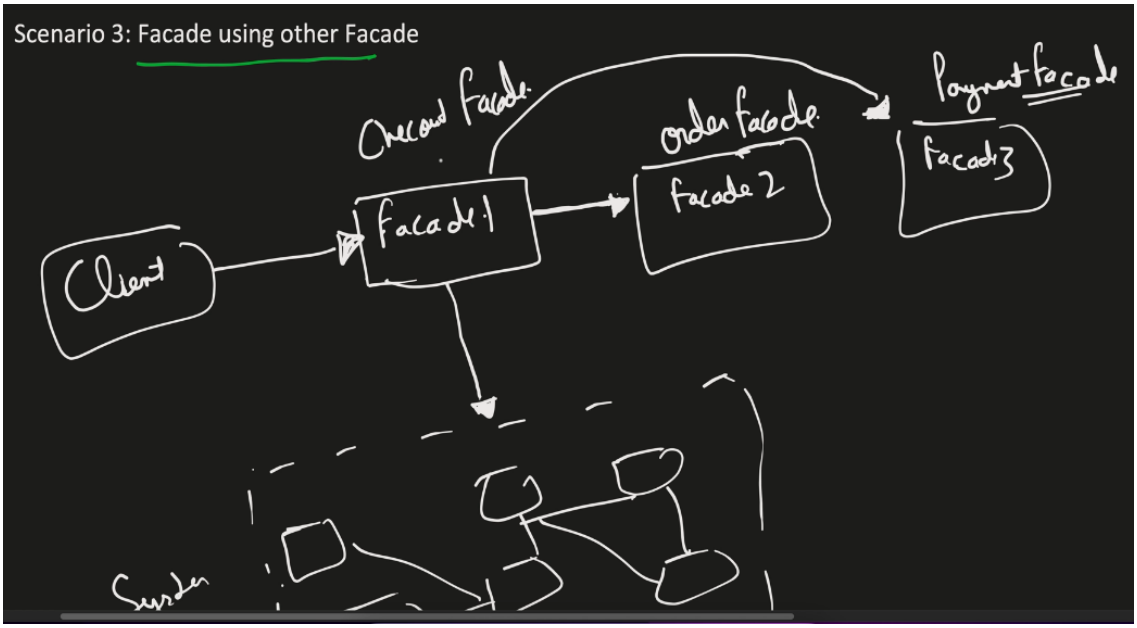
```java
public class OrderFacade {

    ProductDAO productDao;
    Invoice invoice;
    Payment payment;
    SendNotification notification;

    public OrderFacade() {
        productDao = new ProductDAO();
        invoice = new Invoice();
        payment = new Payment();
        notification = new SendNotification();
    }

    public void createOrder() {
```

```java
public class ProductD

    public Product ge
        //get product
        return new Pr
    }
}
```

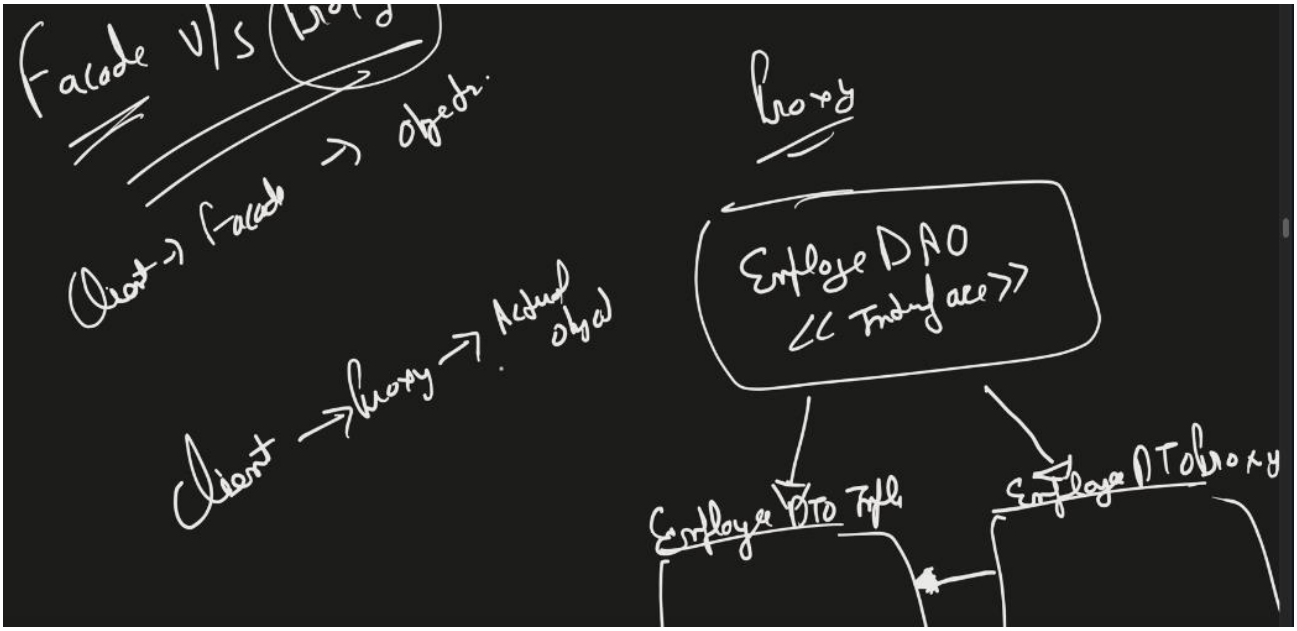
```java
public class Inv

    public void
        //this w
    }
}
```

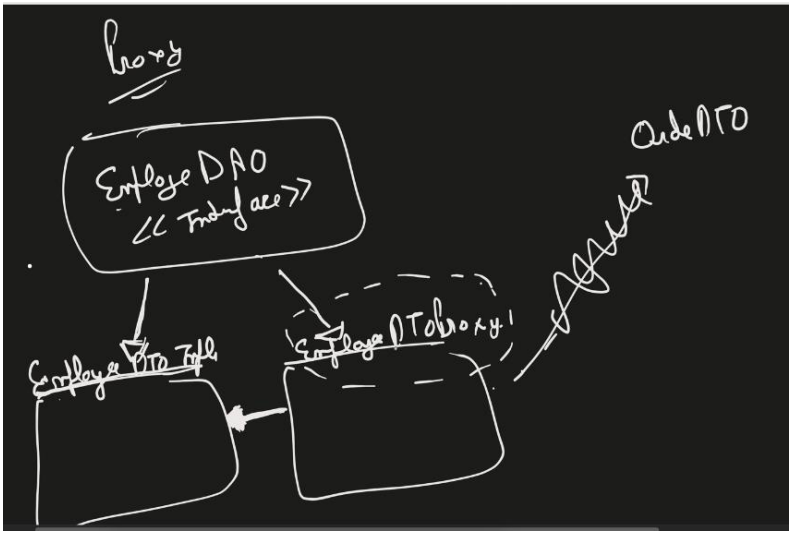
```java
public class OrderFacade {

    ProductDAO productDao;
    Invoice invoice;
    Payment payment;
    SendNotification notification;

    public OrderFacade() {
        productDao = new ProductDAO();
        invoice = new Invoice();
        payment = new Payment();
        notification = new SendNotification();
    }

    public void createOrder() {
        Product product = productDao.getProduct( productId: 121);
        payment.makePayment();
        invoice.generateInvoice();
        notification.sendNotification();
        //order creation successfull
    }
}
```

Case 3 :

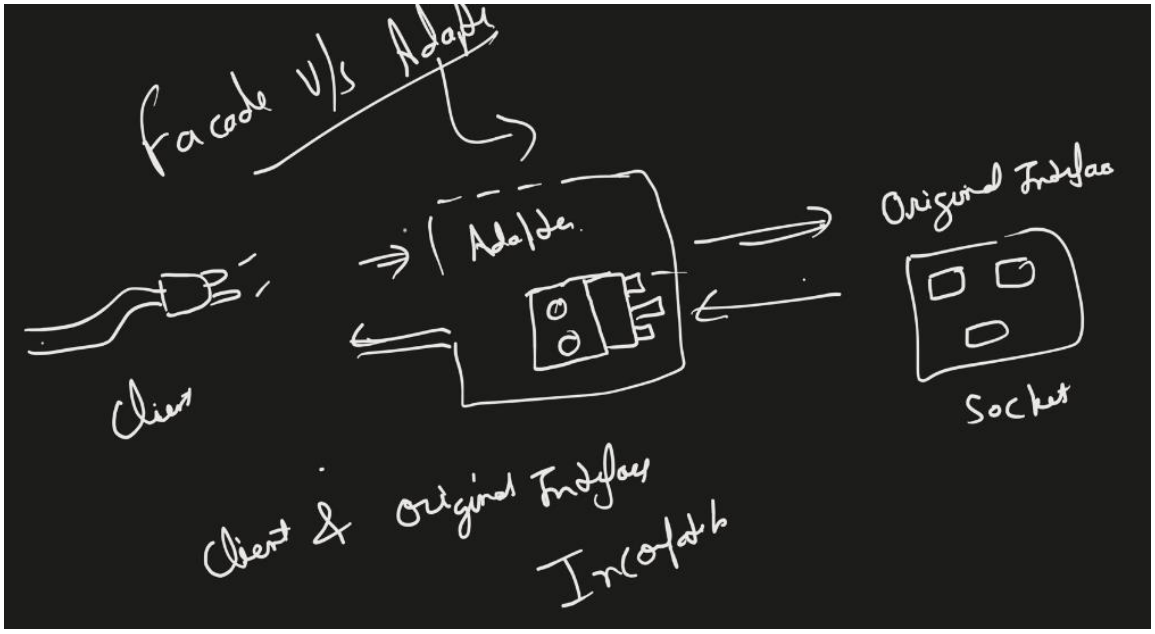


Facade vs Proxy :

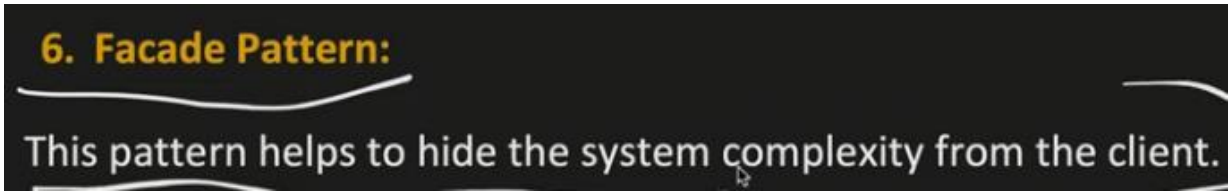


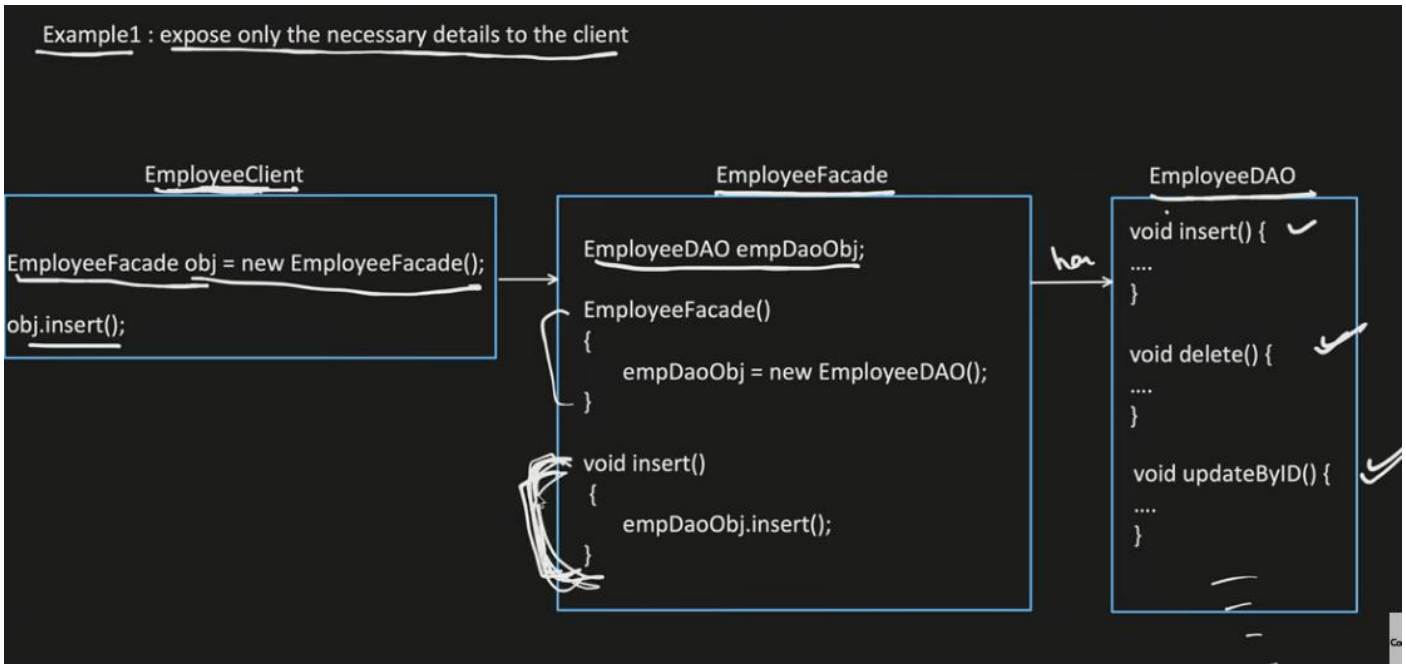


Facade vs Adapter :



https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/structuralpatterns/facade?ref_type=heads





Example2 : hide the system complexity from the client

Client

```
ACFacade obj = new ACFacade();

obj.turnOnButtonClicked();
```

ACFacade

```
ACExternalUnit externalUnit;
ACInternalUnit internalUnit;

ACFacade()
{
    externalUnit = new ACExternalUnit();
    internalUnit = new ACInternalUnit();
}

void turnOnButtonClicked()
{
    internalUnit.acceptUserTempCmd();
    externalUnit.checkVoltage();
    externalUnit.consumeNitrogen();
    externalUnit.startCondensor();
}
```

AC

```
void chec
....
}

void cons
....
}

void start
....
}
```

Some Complex System

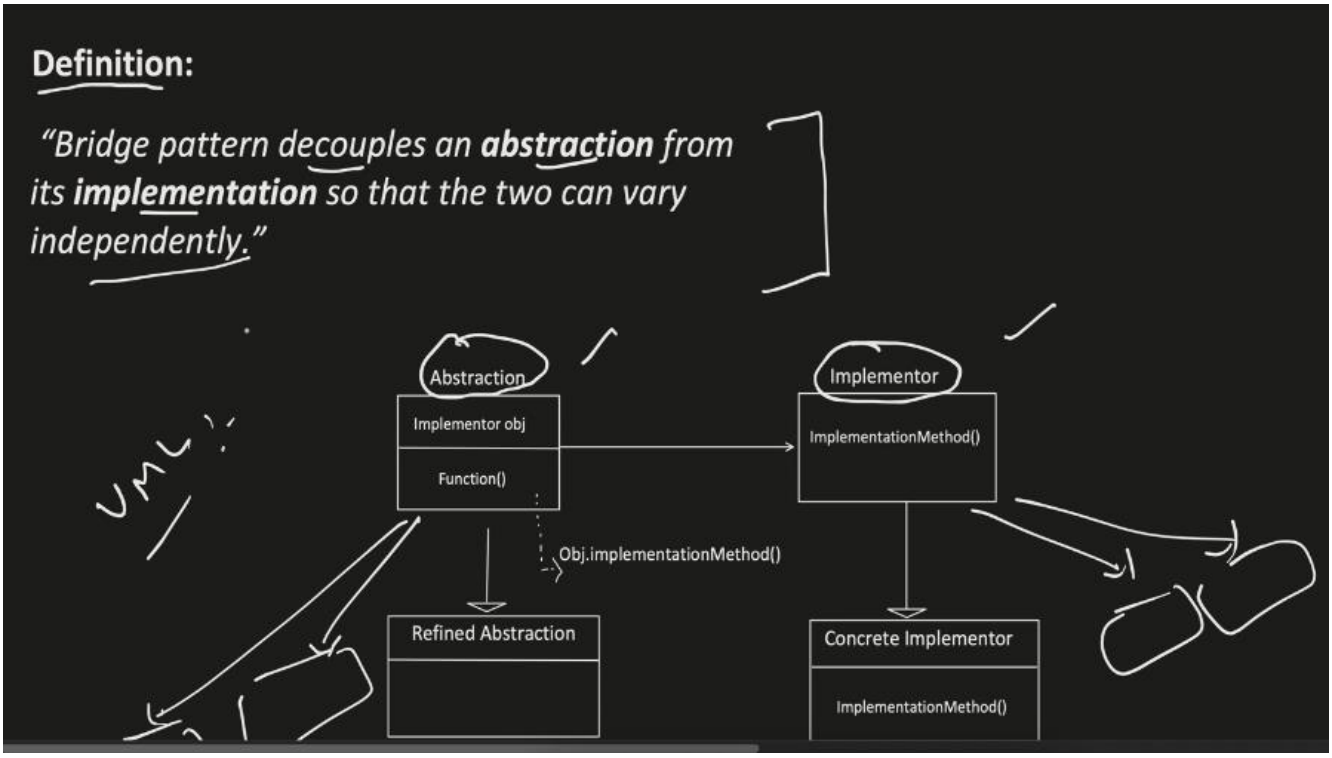
ACExternalUnit

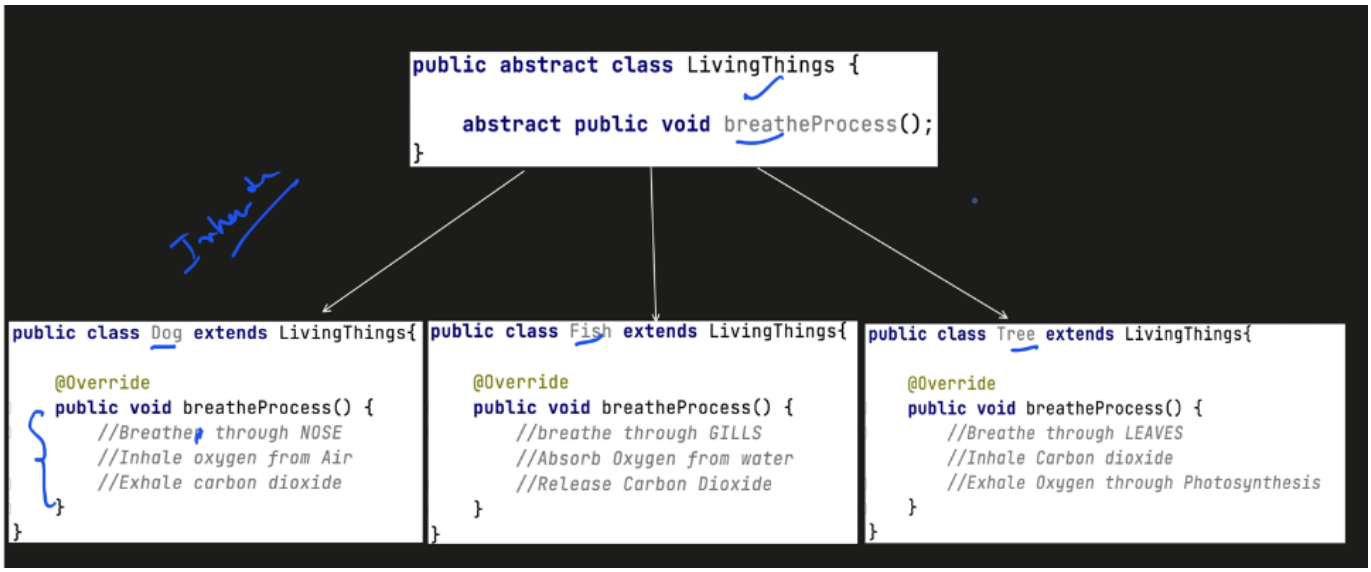
```
void checkVoltage() {
....
}

void consumeNitrogen() {
....
}

void startCondensor() {
....
}
```

ACInternalUnit

```
void acceptUserTempCmd() {

}

void acceptUserOnCmd() {

}

void acceptUserOffCmd() {

}
```

25. Bridge Design Pattern (Structural Design Pattern)





With bridge pattern :

```java
public abstract class LivingThings {

    BreatheImplementor breatheImplementor;

    public LivingThings(BreatheImplementor breatheImplementor){
        this.breatheImplementor = breatheImplementor;
    }
    abstract public void breatheProcess();
}
```

```java
public class Dog extends LivingThings{

    public Dog(BreatheImplementor breatheImplementor) {
        super(breatheImplementor);
    }

    @Override
    public void breatheProcess() {
        breatheImplementor.breathe();
    }
}
```

```java
public class Fish extends LivingThings{

    public Fish(BreatheImplementor breatheImplementor) {
        super(breatheImplementor);
    }

    @Override
    public void breatheProcess() {
        breatheImplementor.breathe();
    }
}
```

```java
public class Tree extends LivingThings{

    public Tree(BreatheImplementor breatheImplementor) {
        super(breatheImplementor);
    }

    @Override
    public void breatheProcess() {
        breatheImplementor.breathe();
    }
}
```

```java
public interface BreatheImplementor {

    public void breathe();
}
```

```java
public class LandBreatheImplementation
        implements BreatheImplementor{

    @Override
    public void breathe() {
        //Breather through NOSE
        //Inhale oxygen from Air
        //Exhale carbon dioxide
    }
}
```

```java
public class WaterBreatheImplementation
        implements BreatheImplementor{

    @Override
    public void breathe() {
        //breathe through GILLS
        //Absorb Oxygen from water
        //Release Carbon Dioxide
    }
}
```

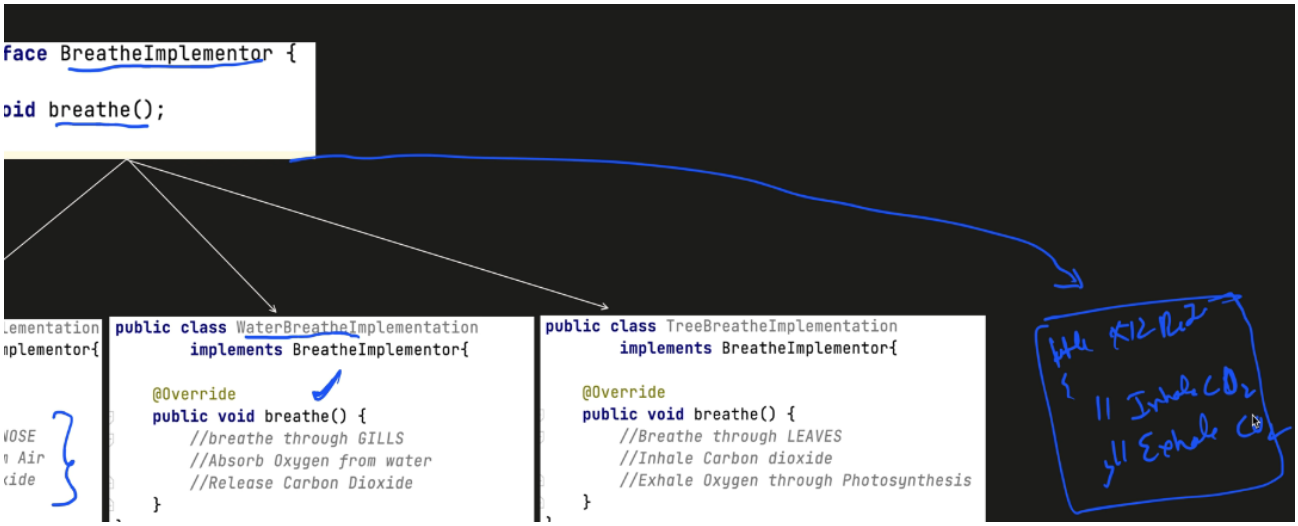
```java
public class TreeBreatheImplementation
        implements BreatheImplementor{

    @Override
    public void breathe() {
        //Breathe through LEAVES
        //Inhale Carbon dioxide
        //Exhale Oxygen through Photosynthesis
    }
}
```

How Client going to invoke for example:

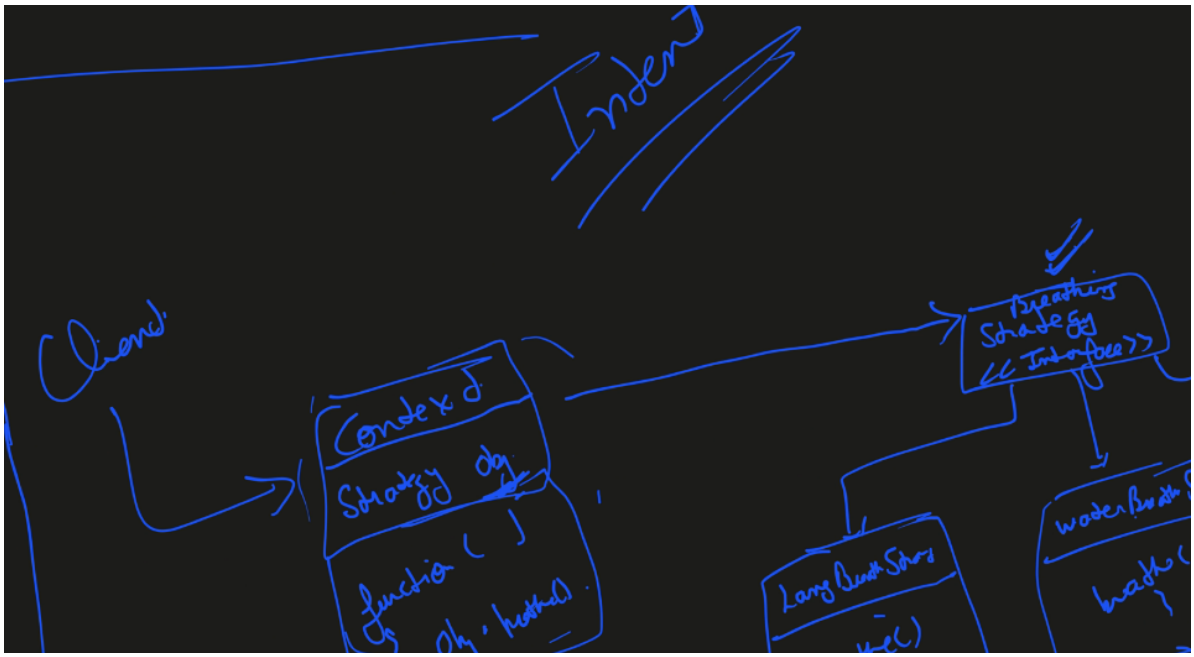
```java
LivingThings fishObject = new Fish(new WaterBreatheImplementation());
fishObject.breatheProcess();
```

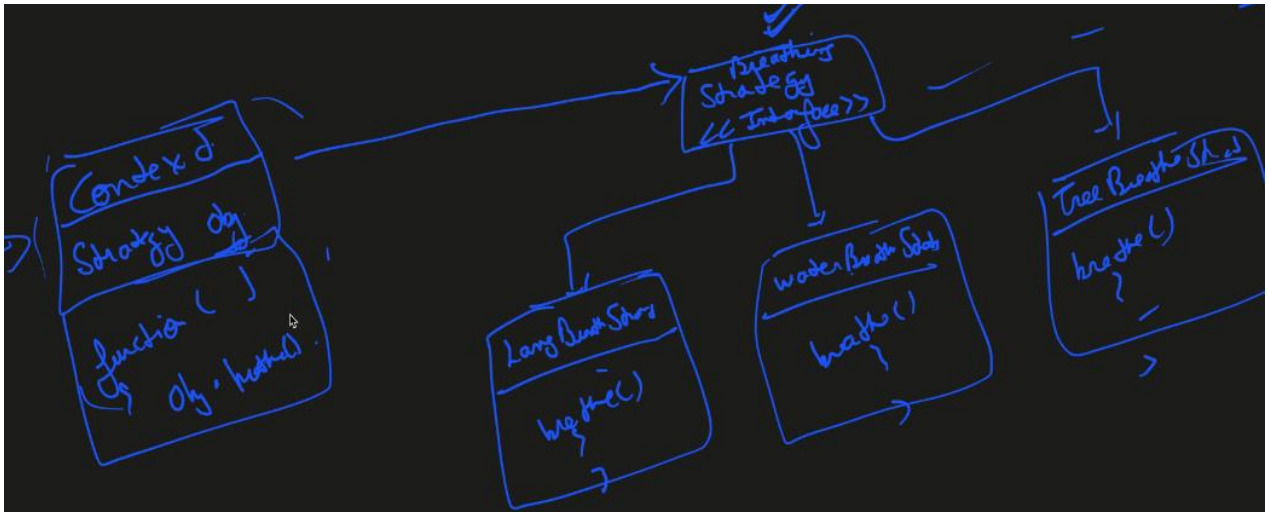
https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/structuralpatterns/bridge?ref_type=heads

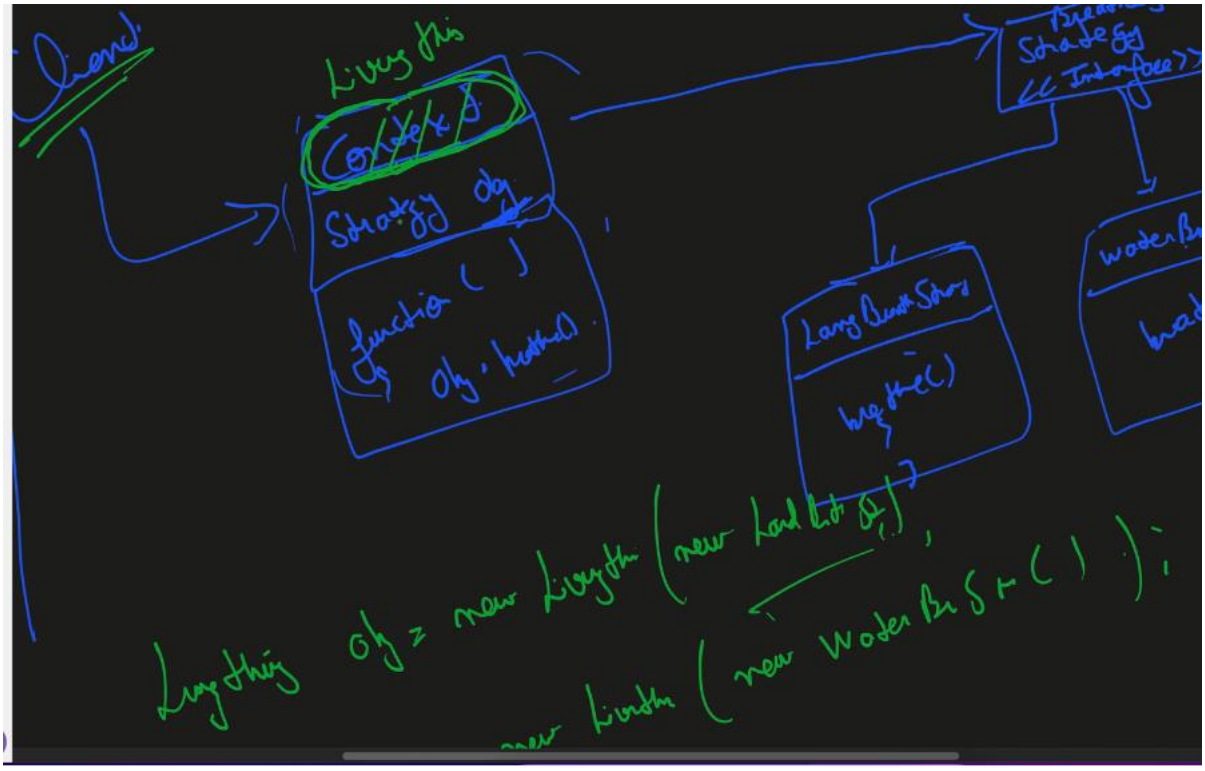
Advantage :



Bridge vs strategy:



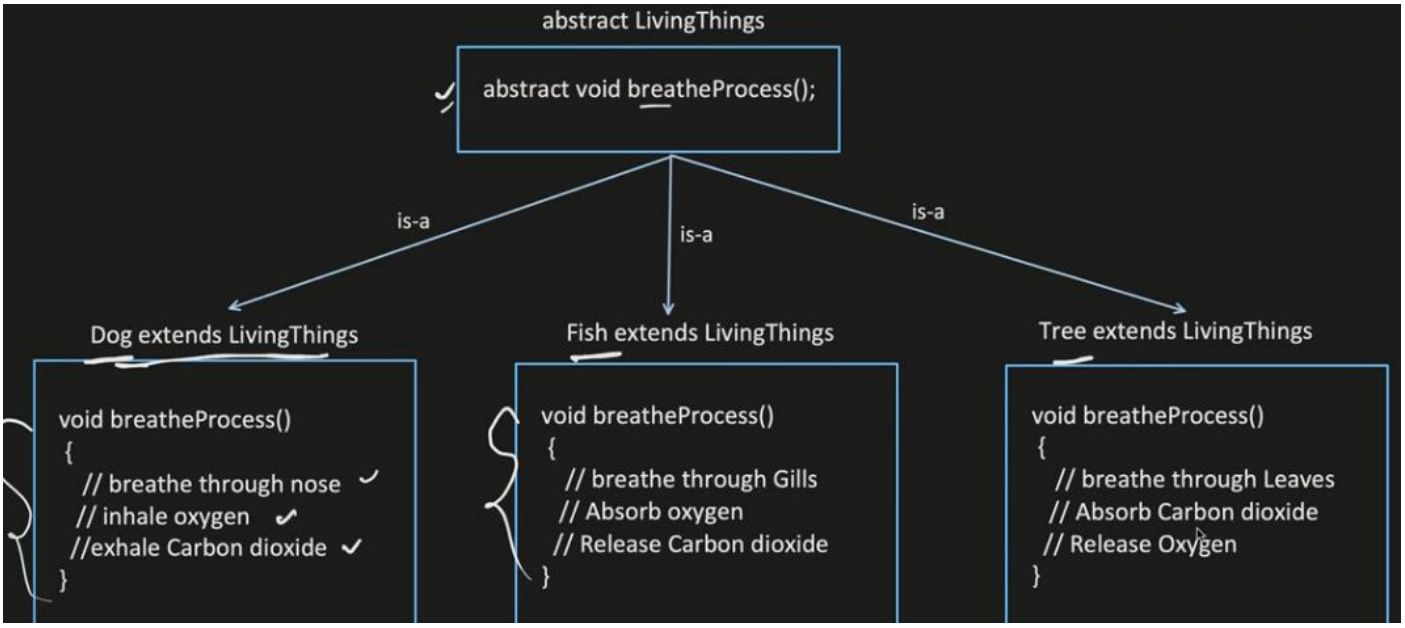




Bridge is a structural pattern designed to separate abstraction from implementation so both can vary independently, while **Strategy** is a behavioral pattern aimed at swapping algorithms or behaviors at runtime

5. Bridge Pattern:

This pattern helps to decouple an abstraction from its implementation, so that two can vary independently.



How to add new Breathing Process, Without adding any class of LivingThings?

abstract LivingThings

```
BreatheImplementor breatheImplementor;

LivingThings(BreatheImplementor breatheImplementor)
{
        this.breatheImplementor = breatheImplementor
}
abstract void breatheProcess();
```

is-a | is-a | is-a

Dog extends LivingThings | Fish extends LivingThings | Tree extends LivingThin

is-a | is-a

Dog extends LivingThings

```
Dog(BreatheImplementor breatheobj)
{
  super(breatheobj);
}

void breatheProcess()
 {

breatheImplementor.breatheProcess();

}
```

Fish extends LivingThings

```
Fish(BreatheImplementor breatheobj)
{
  super(breatheobj);
}

void breatheProcess()
 {

breatheImplementor.breatheProcess();

}
```

Tree extends LivingThing

```
Tree(BreatheImplementor brea
{
   super(breatheobj);
}

void breatheProcess()
 {

breatheImplementor.breatheP

}
```

Concept&&Coding

interface BreatheImplementor

```
void breatheProcess();
```

has-a

is-a | is-a | is-a

LandBreathe implements BreatheImplementor | WaterBreathe implements BreatheImplementor

Concept&&Coding

LandBreathe implements BreatheImplementor

```
void breatheProcess()
 {
   // breathe through nose
  // inhale oxygen
  //exhale Carbon dioxide
 }
```

WaterBreathe implements BreatheImplementor

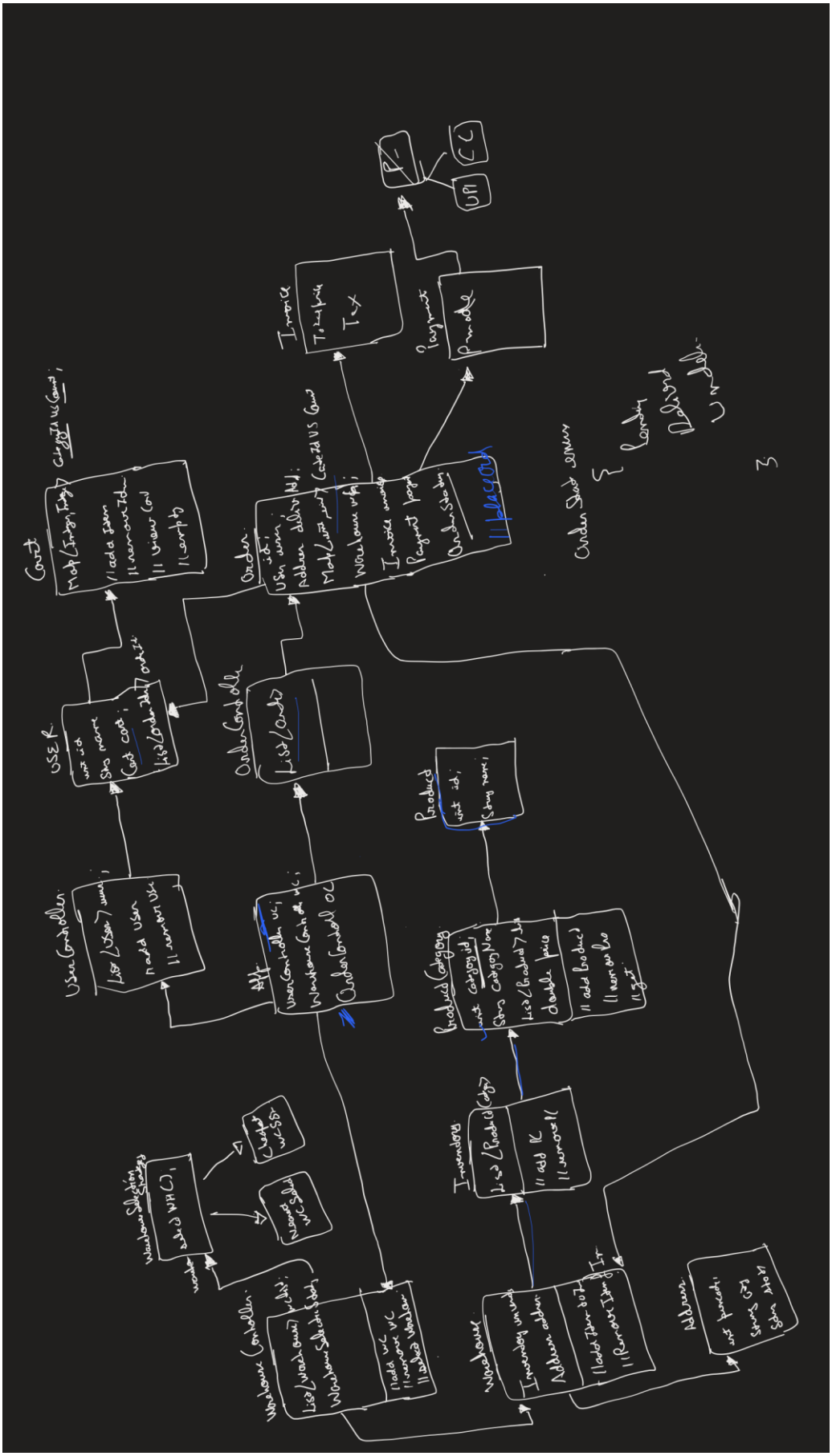
```
void breatheProcess()
 {
   // breathe through Gills
  // Absorb oxygen
  // Release Carbon dioxide
 }
```

TreeBreathe implements BreatheImplementor

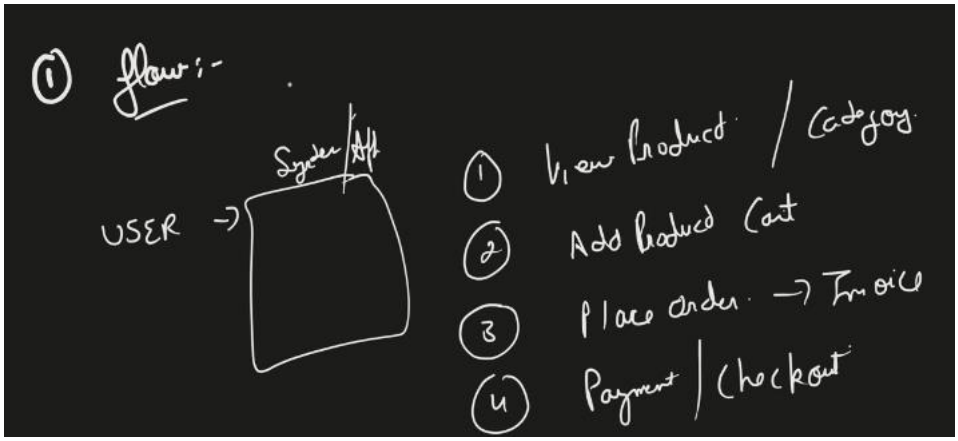
```
void breatheProcess()
 {
   // breathe through Leaves
  // Absorb Carbon dioxide
  // Release Oxygen
 }
```

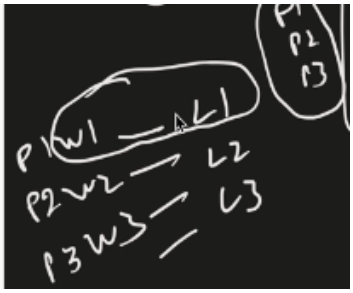
```
LivingThings fishObj= new Fish(new WaterBreathe());
fishObj.breatheProcess();
```

26. LLD of Inventory Management System

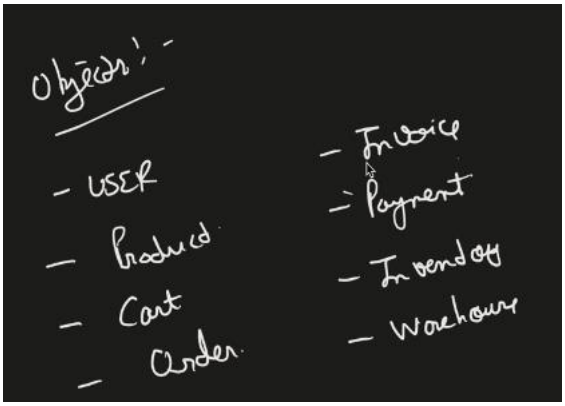


https://gitlab.com/shrayansh8/interviewcodingpractise/-/tree/main/src/main/java/com/conceptandcoding/LowLevelDesign/DesignOrderManagementSystem?ref_type=heads





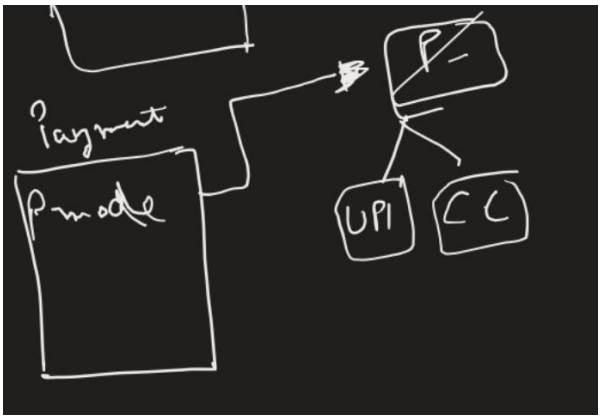
All the products that a user selects will be all from a single warehouse at a particular location

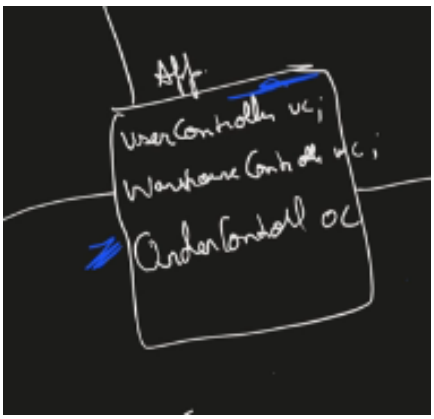


Inventory or stock is a quantity of the goods,products and materials

UML :

https://gitlab.com/shrayansh8/interviewcodingpractise/-/blob/main/src/main/java/com/conceptandcoding/LowLevelDesign/DesignOrderManagementSystem/UML.png?ref_type=heads





https://gitlab.com/shrayansh8/interviewcodingpractise/-/tree/main/src/main/java/com/conceptandcoding/LowLevelDesign/DesignOrderManagementSystem?ref_type=heads

27. Flyweight Design Pattern (Structural Design Pattern)

https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/structuralpatterns/flyweight?ref_type=heads

Two important Interview questions:

- Design Word Processor? ★★ Text Editor
- Design Game?

Definition:

It's one of the Structural Design Pattern.

This Patten helps to reduce the memory usage by sharing data among multiple objects.

GAMING :

Let's first see the issue for Gaming Scenario:

```
public class Robot {

    int coordinateX;
```

```
public class Sprites {
}
```

So chad

```
public class Robot {

    int coordinateX;    // 4B
    int coordinateY;    // 4B
    String type;        // 50 B
    Sprites body; //small 2d bitmap (graphic element)   // 30KB

    Robot(int x, int y, String type, Sprites body){

        this.coordinateX = x;
        this.coordinateY = y;
        this.type = type;
        this.body = body;
    }

    //getter and setters
}
```

```
public class Sprites {
}
```

≈ 31 KB

QuickTime Player

10 Lakh × 31KB

= 31GB

Observe and understand, when to use this pattern:

---

1. When Memory is Limited.
2. When Objects shared data.
   - Intrinsic data : shared among objects and remain same once defined one value.
   - Extrinsic data : changes based on client input and differs from one object to another.
- Creation of Object is expensive.

```
int coordinateX;
int coordinateY;
String type;
Sprites body; //small 2d bitmap
```

Extrinsic

Intrinsic.

This is How it solves the issue:

- From Object, remove all the Extrinsic data and keep Intrinsic Data(this object called Flyweight Object)
- This Flyweight Class can be immutable.
- Extrinsic Data can be passed to the Flyweight class in method parameter.
- Once the Flyweight Object is created, it is Cached and reused whenever required.

```
public interface IRobot {

    public void display(int x, int y);
}
```

```
public class HumanoidRobot implements IRobot {

    private String type;
    private Sprites body; //small 2d bitmap (graphic element)

    HumanoidRobot(String type, Sprites body){
        this.type = type;
        this.body = body;
```

```
public class RoboticDog implements IRobot{

    private String type;
    private Sprites body; //small 2d bitmap

    RoboticDog(String type, Sprites body){
        this.type = type;
```

Map

Humanoid | Robotic

```
public class Main {

    public static void main(String args[]){

        IRobot humanoidRobot1 = RoboticFactory.createRobot( robotType: "HUMANOID");
        humanoidRobot1.display( x: 1, y: 2);

        IRobot humanoidRobot2 = RoboticFactory.createRobot( robotType: "HUMANOID");
        humanoidRobot2.display( x: 10, y: 30);

        IRobot roboDog1 = RoboticFactory.createRobot( robotType: "ROBOTICDOG");
        roboDog1.display( x: 2, y: 9);
```

https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/structuralpatterns/flyweight/roboticgame?ref_type=heads

https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/structuralpatterns/flyweight/wordprocessor?ref_type=heads

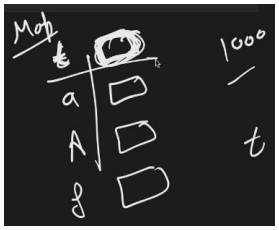
WORD PROCESSOR :

```java
public class Main {

    public static void main(String args[]){

        /* 12345
           This is the data we want to write into the word processor.
           Total = 58 characters
           t = 7 times.
           h = 3 times
           a = 3 times and so on...

        */

        Character object1 = new Character( character: 't',  fontType: "Arial",  size: 10,  row: 0,  column: 0);
        Character object2 = new Character( character: 'h',  fontType: "Arial",  size: 10,  row: 0,  column: 1 );
        Character object3 = new Character( character: 'i',  fontType: "Arial",  size: 10,  row: 0,  column: 2 );
        Character object4 = new Character( character: 's',  fontType: "Arial",  size: 10,  row: 0,  column: 3 );
```

```java
public interface ILetter {

    public void display(int row, int column);
}
```

```java
public class DocumentCharacter implements ILetter{

    private char character;
    private String fontType;
    private int size;

    DocumentCharacter(char character, String fontType, int size){
        this.character = character;
```

```java
public class LetterFactory{

    private static Map<Character, ILetter> characterCache = new HashMap<>();

    public static ILetter crateLetter(char characterValue){

        if(characterCache.containsKey(characterValue)){
            return characterCache.get(characterValue);
        }
        else {

            DocumentCharacter characterObj = new DocumentCharacter(characterValue,  fontType: "Arial",  size: 10);
            characterCache.put(characterValue, characterObj);
            return characterObj;
        }
    }
}
```



```
/*
    this is the data we want to write into the word processor

    Total = 58 characters
    t = 7 times
    h = 3 times
    a = 3 times and so on...

 */

ILetter object1 = LetterFactor.crateLetter( characterValue: 't');
object1.display( row: 0, column: 0);

ILetter object2 = LetterFactor.crateLetter( characterValue: 't');
object2.display( row: 0, column: 6);
```

7. Flyweight Pattern:

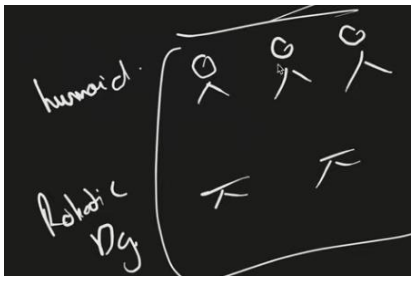
This pattern helps to reduce memory usage by sharing data among multiple objects.

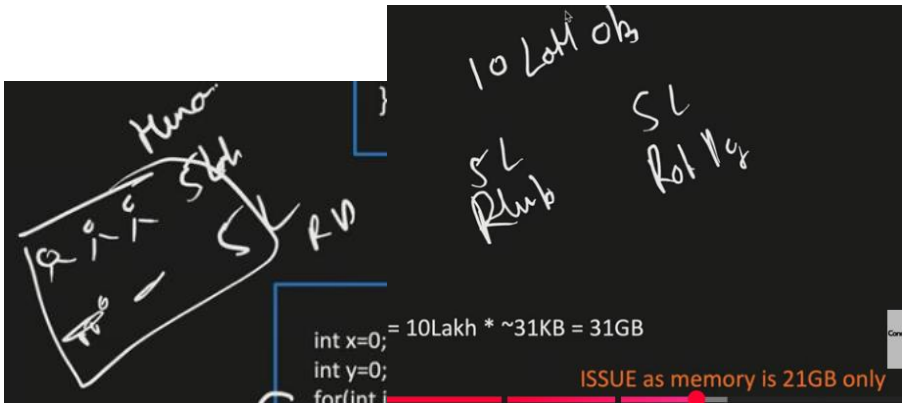
Issue: lets say memory is 21GB

Robot

```
int coordinateX;        //4bytes
Int corrdinateY;        //4bytes
String type;            //50bytes (1bytes * 50 char length)
Sprites body;           //2d bitmap, 31KB

Robot(int x, int y, String type, Sprites body)
{
    this.coordinateX = x;
    this.coordinateY = y;
    this.type = type;
    this.body = body;
}
```

= ~31KB



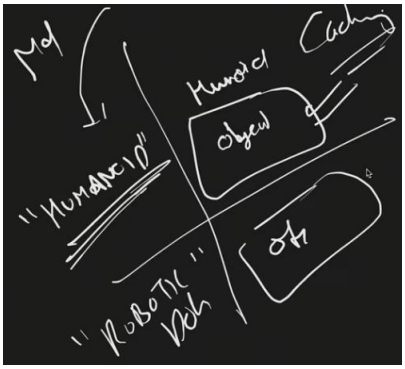


Intrinsic data: shared among objects and remain same once defined one value.
Like in above example : Type and Body is ***Instrinsic*** data.

Extrinsic data: change based on client input and differs from one object to another.
Like in above example: X and Y axis are ***Extrinsic*** data

- From Object, remove all the Extrinsic data and keep only Intrinsic data (this object is called Flyweight Object)
- Extrinsic data can be passed in the parameter to the Flyweight class.
- Caching can be used for the Flyweight object and used whenever required.

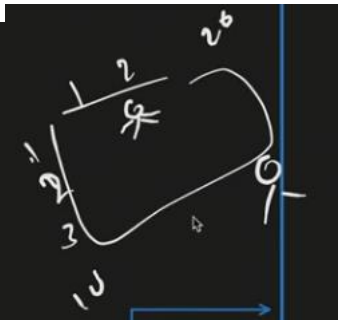
```
interface IRobot
    void display(int x, int y);

HumanoidRobot implements IRobot
    String type;
    Sprites body;  //small 2d bitmap

    Humanoid(String type, Sprites body)
    {

RoboticDog implements IRobot
    String type;
    Sprites body;  //small 2d bitmap

    RoboticDog(String type, Sprites
    {
```




```
HumanoidRobot implements IRobot
    String type;
    Sprites body;//small 2d bitmap

    Humanoid(String type, Sprites body)
    {
        this.type = type;
        this.body = body;
    }

    void display(int x, int y)
    {
        //use the object to render at x, y axis
```

```
RoboticFactory

static Map<String, IRobot> roboticObjectCache = new HashMap<>();

static IRobot createRobot(String robotType)
{
    if(roboticObjectCache.containsKey(robotType))
```

```
IRobot humanoidRobot1 = RoboticFactory.createRobot("HUMANOID");
humanoidRobot .display(1, 2);

IRobot humanoidRobot2 = RoboticFactory.createRobot("HUMANOID");
humanoidRobot2 .display(10, 20);
```



28. Command Design Pattern (Behavioral Design Pattern)

Pattern Category: It's a behavioral pattern.

Lets take the use-case of Remote control which can control various home appliances and with that lets understand the problem, then we will go with this pattern.

Problem :

client

```java
public class Main {
    public static void main(String[] args) {

        AirConditioner ac = new AirConditioner();
        ac.turnOnAC();
        ac.setTemperature(24);
        ac.turnOffAC();
    }
```

```java
public class AirConditioner {

    boolean isOn;
    int temperature;

    public void turnOnAC(){
        isOn = true;
        System.out.println("AC is ON");
    }

    public void turnOffAC(){
        isOn = false;
        System.out.println("AC is OFF");
    }

    public void setTemperature(int temp){
        this.temperature = temp;
        System.out.println("Temperature changed to:" + temperature) ;
    }
}
```

Problem with above implementation:

- Lack of Abstraction :
Today, process of turning on AC is simple, but if there are more steps, client has to aware all of that, which is not good.

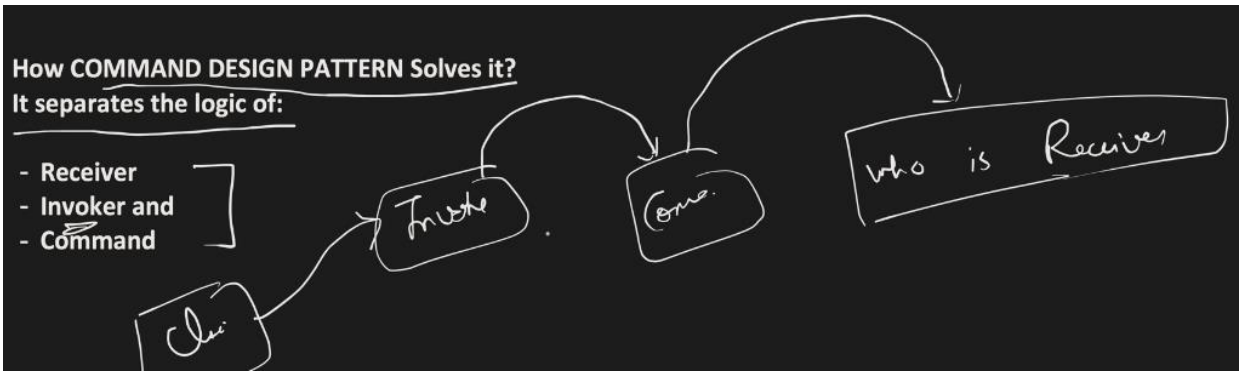
- Undo/Redo Functionality:
What if I want to implement the undo/redo capability. How it will be handled.

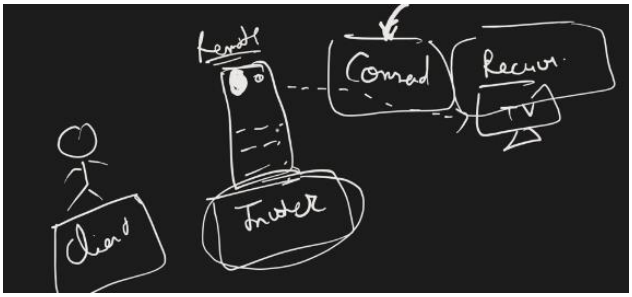
- Difficulty in Code Maintenance:
What if in future, we have to support more commands for more devices example Bulb. Ma

```java
public class Main {
    public static void main(String[] args) {

        AirConditioner ac = new AirConditioner();
        ac.turnOnAC();
        ac.setTemperature(24);
        ac.turnOffAC();

        Bulb bulbObj = new Bulb();
        bulbObj.turnOnBulb();
        bulbObj.turnOffBulb();
    }
}
```

```java
public class Bulb {

    boolean isOn;

    public void turnOnBulb(){
        isOn = true;
        System.out.println("Bulb is ON");
    }

    public void turnOffBulb(){
        isOn = false;
        System.out.println("Bulb is OFF");
    }
}
```

```java
public class AirConditioner {

    boolean isOn;
    int temperature;

    public void turnOnAC(){
        isOn = true;
        System.out.println("AC is ON"
    }

    public void turnOffAC(){
        isOn = false;
        System.out.println("AC is OFF
    }

    public void setTemperature(int te
        this.temperature = temp;
        System.out.println("Temperatu
    }
}
```





Invoker

```java
public class MyRemoteControl {

    ICommand command;
    MyRemoteControl(){
    }

    public void setCommand(ICommand command){
        this.command = command;
    }

    public void pressButton() {
        command.execute();
    }
}
```

Command

```java
public interface ICommand {

    public void execute();
}
```

```java
COnCommand implements ICommand{
```

```java
public class TurnACOffCommand implements ICommand{
```

Receiver

```java
public class AirConditioner {

    boolean isOn;
    int temperature;

    public void turnOnAC(){
        isOn = true;
        System.out.println("AC is ON");
    }

    public void turnOffAC(){
        isOn = false;
        System.out.println("AC is OFF");
    }

    public void setTemperature(int temp){
        this.temperature = temp;
        System.out.println("Temperature changed to:" + temperature) ;
    }
```

Client

```java
public class Main {
    public static void main(String[] args) {

        //AC object
        AirConditioner airConditioner = new AirConditioner();

        //remote
        MyRemoteControl remoteObj = new MyRemoteControl();

        //create the command and press the button
        remoteObj.setCommand(new TurnACOnCommand(airConditioner));
        remoteObj.pressButton();
    }
}
```

https://gitlab.com/shrayansh8/lld-lowleveldesign/-/tree/main/src/main/java/com/conceptcoding/behavioralpatterns/command?ref_type=heads

7. **Command Pattern:** Turns requests (commands) into objects, allowing you to either para ither parametrized or queue them.
This will help to decouple the request Sender and receiver.

Undo redo :

```java
import java.util.Stack;

public class MyRemoteControl {

    Stack<ICommand> acCommandHistory = new Stack<>();
    ICommand command;

    MyRemoteControl(){
    }

    public void setCommand(ICommand command){
        this.command = command;
    }

    public void pressButton()
    {
        command.execute();
        acCommandHistory.add(command);
    }

    public void undo(){

        if(!acCommandHistory.isEmpty()){
            ICommand lastCommand = acCommandHistory.pop();
            lastCommand.undo();
        }
    }
}
```

```java
public interface ICommand {

    public void execute();
    public void undo();

}
```

```java
public class TurnACOnCommand implements ICommand{

    AirConditioner ac;
    TurnACOnCommand(AirConditioner ac){
        this.ac = ac;
    }

    @Override
    public void execute() {
        ac.turnOnAC();
    }

    @Override
    public void undo() {
        ac.turnOffAC();
    }
```

```java
public class TurnACOffCommand implements ICommand{

    AirConditioner ac;
    TurnACOffCommand(AirConditioner ac){
        this.ac = ac;
    }

    @Override
    public void execute() {
        ac.turnOffAC();
    }

    @Override
    public void undo() {
        ac.turnOnAC();
    }
```

```java
lic interface ICommand {

 public void execute();
 public void undo();
```

```java
public class AirConditioner {

    boolean isOn;
    int temperature;

    public void turnOnAC(){
        isOn = true;
        System.out.println("AC is ON");
    }

    public void turnOffAC(){
        isOn = false;
        System.out.println("AC is OFF");
    }

    public void setTemperature(int temp){
        this.temperature = temp;
        System.out.println("Temperature changed to:" + temperature) ;
    }
}
```

```java
ommand{
```

```java
public class TurnACOffCommand implements ICommand{

    AirConditioner ac;
    TurnACOffCommand(AirConditioner ac){
        this.ac = ac;
    }

    @Override
    public void execute() {
        ac.turnOffAC();
    }

    @Override
    public void undo() {
        ac.turnOnAC();
    }
}
```

```java
import java.util.Stack;

public class MyRemoteControl {

    Stack<ICommand> acCommandHistory = new Stack<>();
    ICommand command;

    MyRemoteControl(){
    }

    public void setCommand(ICommand command){
        this.command = command;
    }

    public void pressButton()
    {
        command.execute();
        acCommandHistory.add(command);
    }

    public void undo(){

        if(!acCommandHistory.isEmpty()){
            ICommand lastCommand = acCommandHistory.pop();
            lastCommand.undo();
        }
    }
}
```

```java
public class Main {
    public static void main(String[] args) {

        //AC object
        AirConditioner airConditioner = new AirConditioner();

        //remote
        MyRemoteControl remoteObj = new MyRemoteControl();

        //create the command and press the button
        remoteObj.setCommand(new TurnACOnCommand(airConditioner));
        remoteObj.pressButton();

        //undo the last operation
        remoteObj.undo();
    }
```

Problem with below code:

process of turning on AC is simple, but if there are more steps, client has t client has to aware all of that, which is not good. So Sender and Receiver are not decoupled.

```
d main(String[] args) {

r ac = new AirConditioner();
;
ture(24);
);
```

```
public class AirConditioner {

    boolean isOn;
    int temperature;

    public void turnOnAC(){
        isOn = true;
        System.out.println("AC is ON");
    }

    public void turnOffAC(){
        isOn = false;
        System.out.println("AC is OFF");
    }

    public void setTemperature(int temp){
```